

An
Industrial Training Report
On
MERN Stack
Submitted partial fulfillment for the award of degree
of
Bachelor of Technology
In
Computer Science and Engineering



Submitted to:

Dr. Pradeep Jha

HOD-CSE/IT/AI&DS/IOT/CYBER SECURITY Dept.

Submitted By:

Kanchan Prajapat

24EGJCS103

Department of Computer Science & Engineering

Global Institute of Technology

Jaipur (Rajasthan)-302022

Session: 2025-26

Certificate



Acknowledgement

I express my sincere gratitude to **Mr. AshaRam Gurjar**, Assistant Professor, Department of Computer Science and Engineering, Global Institute of Technology, Jaipur, for providing me the opportunity to undergo Industrial Training in the domain of **Web Development using the MERN Stack**. His valuable guidance, continuous support, and insightful feedback played a significant role in enhancing my understanding of full-stack development.

I extend my heartfelt thanks to the entire team of **GIT, Jaipur** for their support and encouragement throughout the training period. Their expertise, real-time examples, and project-based mentoring helped me gain practical exposure to modern technologies such as React.js, Node.js, Express.js, and MongoDB.

I am also thankful to **Dr. I. C. Shrama, Principal of GIT**, and **Dr. Pradeep Jha**, Head of the Department of Computer Science and Engineering , for their motivation and academic guidance, which inspired me to complete this industrial training successfully.

Lastly, I express my deep gratitude to my parents, friends, and fellow learners for their constant encouragement, cooperation, and moral support during the entire training duration.

Place: GIT , Jaipur

Kanchan Prajapat

24EGJCS103

B.Tech III Semester, II Year, CSE

Abstract

This industrial training focused on comprehensive full-stack web development using the **MERN Stack (MongoDB, Express.js, React.js, and Node.js)**, following a practical, project-oriented learning approach. The training emphasized the core principles of modern web development, beginning with essential frontend technologies such as **HTML, CSS, and JavaScript**, and gradually advancing toward component-based UI development using **React.js**. I gained hands-on experience in building dynamic and responsive user interfaces, managing component states, using React Hooks, and structuring multi-page applications through React Router.

On the backend, I worked extensively with **Node.js** and **Express.js**, learning how to design RESTful APIs, implement routing, configure middleware, and manage server-side logic effectively. The training also deepened my understanding of **MongoDB**, where I practiced CRUD operations, schema modeling with Mongoose, and handling real-time data flows between the frontend and backend. Key learning outcomes included API integration, secure data handling, asynchronous operations, and building scalable server architectures.

Throughout the program, I developed practical problem-solving skills by working on modular tasks and mini-projects, culminating in the integration of a full MERN workflow. The training enhanced my knowledge of development best practices, including code reusability, clean folder structuring, version control, error handling, and deployment on cloud platforms. This comprehensive experience equipped me with the technical proficiency and confidence required to build complete, interactive, and user-centric web applications using the MERN stack.

Table of Contents

• Certificate.....	ii
• Acknowledgement.....	iii
• Abstract	iv
• Table of Contents	v
• List of Figures	vi
• List of Symbols	vii

Chapters

• Chapter 1: Introduction to Web Development & MERN Stack	08
• Chapter 2: HTML, CSS & JavaScript Fundamentals (Pre-MERN Basics).....	15
• Chapter 3: React.js Basics and Component-Based Development.....	16
• Chapter 4: Introduction to Node.js	21
• Chapter 5: Express.js Routing & REST API Development	26
• Chapter 6: MongoDB & Mongoose	30
• Chapter 7: MERN Integration and Authentication, Security & Deployment....	35
• My Project	41
• Training Summary	45
• Key Outcomes	46
• References	47

List of Figures

• Figure 01: MERN Stack Architecture Overview	03
• Figure 02: Basic Structure of an HTML Document.....	06
• Figure 03: CSS Box Model Diagram.....	07
• Figure 04: Dom Tree Representation.....	08
• Figure 05: Working of React Virtual DOM	11
• Figure 06: Node.js Event Loop Working	16
• Figure 07: Express.js Basic Folder Structure	22
• Figure 08: MongoDB Document Structure Overview	24
• Figure 09: MERN CRUD Data Flow	30
• Snapshot 1	39
• Snapshot 2	39
• Snapshot 3	39

List of Tables

- **Table 01:** Frontend, Backend & Database Responsibilities in MERN..... 02
- **Table 02:** Comparison of Frontend Technologies 09
- **Table 03:** SQL vs NoSQL (MongoDB) Differences.....26

List of Symbols

- **MERN** — MongoDB, Express.js, React.js, Node.js
- **API** — Application Programming Interface
- **DOM** — Document Object Model
- **JSX** — JavaScript XML
- **CRUD** — Create, Read, Update, Delete
- **REST** — Representational State Transfer
- **UI** — User Interface
- **UX** — User Experience
- **DB** — Database
- **JSON** — JavaScript Object Notation
- **HTTP** — Hypertext Transfer Protocol
- **JWT** — JSON Web Token
- **NPM** — Node Package Manager
- **SPA** — Single Page Application
- **MVC** — Model View Controller
- **URL** — Uniform Resource Locator

Chapter 1

Introduction to Web Development & MERN Stack

Today marked the beginning of my industrial training in **Web Development** with a focus on the MERN Stack. The session introduced me to the fundamentals of modern web applications and how client-server communication plays a key role in delivering dynamic user experiences. I learned that full-stack development combines frontend, backend, and database management to build complete solutions. The MERN stack—**MongoDB**, **Express.js**, **React.js**, and **Node.js**—brings all these layers together within a single JavaScript ecosystem. This makes development faster, more efficient, and easier to manage. Overall, the session helped me understand why MERN is widely used for scalable and interactive applications.

Overview of Web Development

Web development involves creating applications that run on browsers and interact with servers. The instructor explained the major areas:

Frontend (Client Side): User interface, layouts, interactions, and design logic.

Backend (Server Side): APIs, authentication, business logic, data processing.

Database: Storage, retrieval, and management of application data.

Understanding the interaction between these layers is crucial for full-stack development.

Why MERN Stack?

The MERN stack is composed of:

- **M — MongoDB**
- **E — Express.js**
- **R — React.js**
- **N — Node.js**

The instructor emphasized the following reasons why MERN is used extensively:

- Same language (JavaScript) for both frontend & backend
- Rapid development with reusable components
- High performance due to non-blocking architecture
- Widely used in modern companies and startups
- Easy deployment on cloud platforms like Render, Vercel, and MongoDB Atlas

Frontend–Backend–Database Responsibilities (MERN Stack)

The following table summarizes the responsibilities of each layer in the MERN architecture.

Layer	Technology Used	Responsibilities
Frontend	React.js	UI design, routing, state management, form handling, API calls, interactive components
Backend	Node.js + Express.js	Business logic, REST APIs, authentication, middleware, server routing, handling requests & responses
Database	MongoDB + Mongoose	Data storage, CRUD operations, collections, schema modeling, indexing

Table 01: Frontend, Backend & Database Responsibilities in MERN

This table helped me visualize how each part contributes to a fully functional application.

MERN Architecture and Workflow

To understand how a MERN application functions from end to end, I explored how data flows between the client (React), server (Node/Express), and database (MongoDB). The instructor explained that a typical MERN application follows this sequence:

1. The user interacts with the **UI built in React**.
2. React sends **API requests** to the **Express.js server**.
3. Express processes the request and interacts with **MongoDB** for data.
4. MongoDB returns the response back to Express.
5. Express sends the final results to the **React frontend**.

This cycle repeats for every user action such as login, signup, viewing data, or submitting forms.

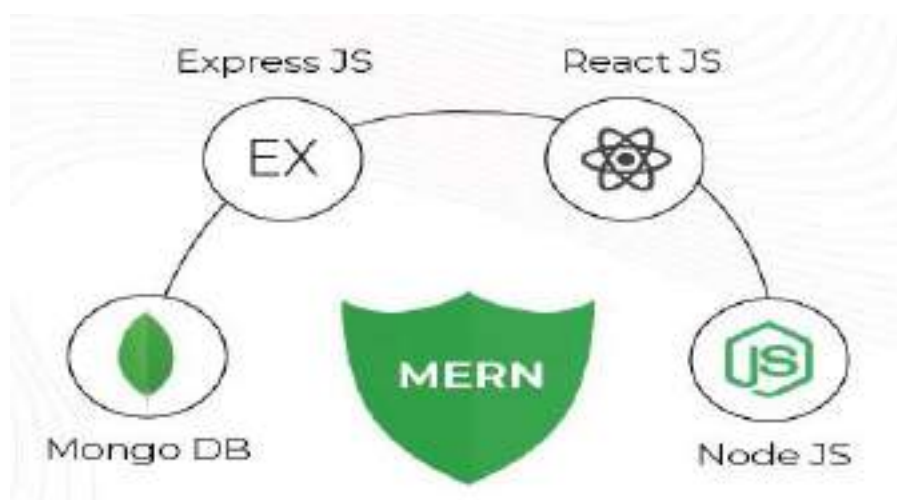


Figure 01: MERN Stack Architecture Overview

This figure will visually represent:

React (Frontend) ↔ Express/Node (Backend Server) ↔ MongoDB (Database)

Development Environment Setup

The training guided me through installing and configuring tools required for MERN development:

Node.js & NPM – For backend and package management

Visual Studio Code – Primary code editor

MongoDB Compass / MongoDB Atlas – For database visualization

Browser Developer Tools – For debugging frontend applications

The folder structure, package.json configuration, and dependency installation (React, Express, Mongoose) were explained in detail.

Key Technologies Introduced

The first day familiarized me with the essential technologies of the MERN stack:

- **React.js Basics** Component structure, JSX syntax and Rendering logic
- **Node.js Fundamentals** JavaScript running outside the browser, Non-blocking I/O and Event-driven architecture
- **Express.js Essentials** Route handling, API creation and Middleware usage
- **MongoDB Concepts** Collections, Documents (BSON format) and Schema-less design

Each of these technologies forms a major part of the training and will be explored deeply in later chapters.

The MERN stack, being one of the most in-demand stacks, is used by companies like Netflix, Uber, Facebook, Shopify, and many startups.

HTML, CSS & JavaScript Fundamentals

The second day of my industrial training focused on strengthening the foundational concepts of frontend development, which form the base of any MERN stack application. Before moving into React.js, it is important to understand how web pages are structured, styled, and made interactive using HTML, CSS, and JavaScript. The instructor emphasized that even when working with advanced frameworks, strong basics lead to cleaner, scalable, and more efficient projects.

Introduction to Frontend Basics

Frontend development is responsible for everything that a user sees and interacts with on the browser. The instructor explained that modern interfaces built using React still rely on core HTML, CSS, and JavaScript concepts.

A **web page** consists of three major components:

HTML – Defines the structure of content

CSS – Styles and formats the content

JavaScript – Adds interactivity and logical behavior

These three technologies together create responsive and dynamic user interfaces.

HTML Structure & Semantic Tags

HTML (HyperText Markup Language) provides the backbone of every website. I learned about the basic structure of an HTML document and the importance of semantic tags for SEO and accessibility.

Common Semantic Tags:

`<header>` , `<nav>`, `<section>`, `<article>`, `<footer>` etc.

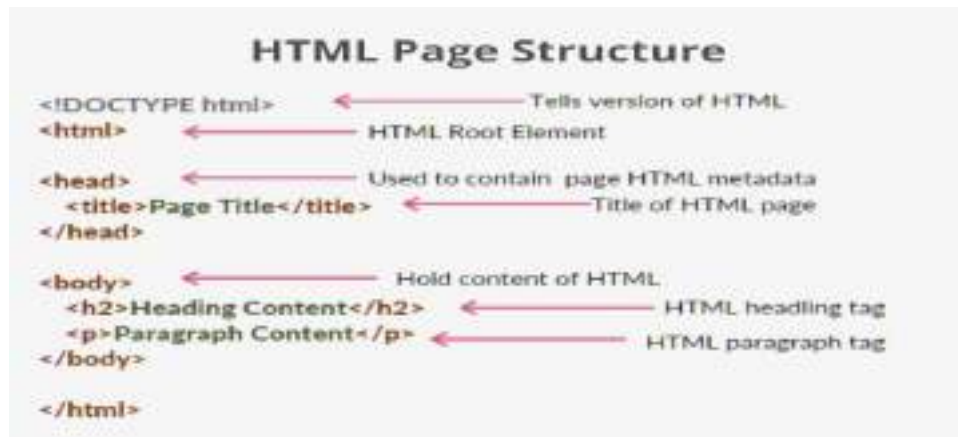


Figure 02: Basic Structure of an HTML Document

This diagram will illustrate:

`<!DOCTYPE html>` → `<html>` → `<head>` → `<body>` hierarchy.

```
<!DOCTYPE html><html><head>
```

```
  <title>My First Web Page</title></head><body>
```

```
<header>
```

```
  <h1>Welcome to MERN Training</h1>
```

```
</header>
```

```
<p>This is my first day learning frontend basics.</p></body>
```

```
</html>
```

CSS Styling – Box Model, Flexbox & Grid

CSS (Cascading Style Sheets) controls the layout, colors, spacing, and responsiveness of a webpage. The trainer introduced:

CSS Box Model

Flexbox for alignment

Grid for complex layouts

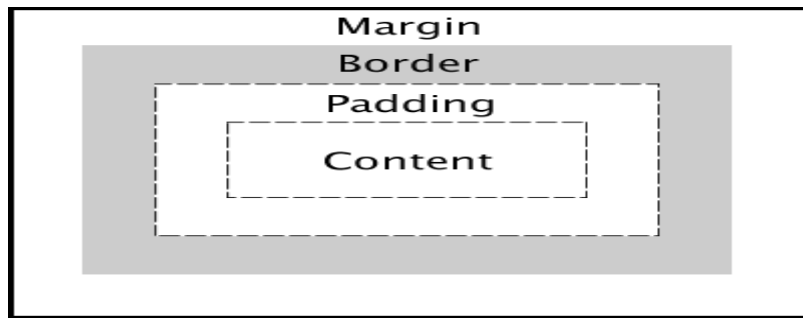


Figure 03: CSS Box Model Diagram

Example Code: Basic CSS Styling

```
body {  
  
    font-family: Arial;  
  
    margin: 20px;  
  
}  
  
.container {  
  
    display: flex;  
  
    gap: 20px;  
  
}  
  
.box {  
  
    width: 200px;  
  
    padding: 15px;  
  
    background-color: lightblue;  
  
    border-radius: 8px;  
  
}
```

JavaScript ES6+ Features

JavaScript is the heart of interactivity in websites. Before learning React, I studied important ES6+ concepts such as:

- let and const
- Arrow functions
- Template literals
- Spread & Rest operators
- Array methods (map, filter, reduce)

These features make code cleaner and more efficient, especially in React.

Example Code: Arrow Function & Template Literal

```
const greet = (name) => {
    return `Hello, ${name}! Welcome to MERN training.`;
}
console.log(greet("Kanchan"));
```

DOM (Document Object Model) Manipulation

DOM is the browser's way of representing an HTML document as a tree structure. JavaScript can modify the DOM dynamically.

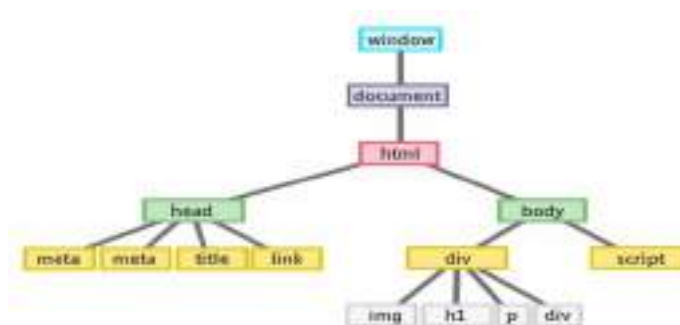


Figure 04: DOM Tree Representation

This figure shows how elements like `<div>`, `<p>`, `<h1>` become nodes of a tree.

Example Code: Changing Text with DOM

```
document.getElementById("title").innerText = "Welcome to MERN Stack";
```

Event Handling in JavaScript

Events allow interaction between the user and the webpage.

Common events:

`onclick`

`onmouseover`

`onkeyup`

Example Code: Button Click Event

```
function showMessage() {  
    alert("Button clicked!");  
}
```

```
<button onclick="showMessage()">Click Me</button>
```

Table: Comparison of HTML, CSS & JavaScript

Technology	Purpose	Example Usage
HTML	Structure of webpage	Forms, headings, layouts
CSS	Styling & layout	Colors, spacing, animations
JavaScript	Interactivity & logic	Validations, API calls, dynamic DOM

Table 02: Comparison of Frontend Technologies

This table helped me understand how each technology plays a unique role in frontend development.

React.js Basics and Component-Based Development

Today my industrial training introduced me to **React.js**, one of the most powerful and widely used JavaScript libraries for building modern user interfaces. The instructor explained that while HTML, CSS, and JavaScript handle the foundation of the web, React enables developers to create dynamic and reusable components, making large-scale application development easier and more efficient.

React is used by top companies such as Facebook, Instagram, Netflix, Uber, and Airbnb due to its performance, modularity, and community support.

Introduction to React.js

React.js is a **component-based JavaScript library** created by Facebook. Unlike traditional web development where pages reload on every action, React implements a dynamic UI using its efficient rendering mechanism.

Key characteristics of React:

- Component-driven architecture
- Virtual DOM for fast updates
- Reusability and modular structure
- Unidirectional data flow
- Works seamlessly with REST APIs

React enables applications to update only the required parts of the UI instead of reloading the entire page.

Virtual DOM Explained

One of the biggest reasons React is fast is due to its **Virtual DOM (Document Object Model)**. Instead of manipulating the actual DOM directly, React creates an in-memory representation (virtual DOM) and updates only the changed elements.



Figure 05: Working of React Virtual DOM

This optimized update process improves performance significantly, especially in interactive applications.

Components, Props & State

React applications are built using **components**, which are independent and reusable pieces of UI.

Types of Components:

1. **Functional Components**
2. **Class Components** (less used today)

Props

Props are read-only inputs used to pass data between components.

State

State stores the mutable data of a component and triggers a UI update when changed.

React Hooks – useState and useEffect

React Hooks allow functional components to manage state and lifecycle events.

useState Example

```
import React, { useState } from 'react';

function Counter() {

  const [count, setCount] = useState(0);

  return (

    <div>

      <h2>Count: {count}</h2>

      <button onClick={() => setCount(count + 1)}>

        Increase

      </button>

    </div>

  );

}

export default Counter;
```

useEffect Example

```
useEffect(() => {

  console.log("Component loaded!");

}, []);
```

Conditional Rendering

React allows rendering content based on conditions. This helps in showing messages, loading screens, or different UI elements dynamically.

Example: Conditional Rendering

```
function Welcome({ isLoggedIn }) {
```

```

return (
  <div>
    {isLoggedIn ? <h3>Welcome User</h3> : <h3>Please Login</h3>}
  </div>
);
}

```

Lists, Keys, and Basic Forms

Lists help display repeated content. Keys ensure React efficiently identifies each item.

Example: Rendering a List

```

const names = ["Raj", "Sita", "Aman"];

function NameList() {
  return (
    <ul>
      {names.map((n, index) => (
        <li key={index}>{n}</li>
      ))}
    </ul>
  );
}

```

Example: Simple Controlled Form

```

function Form() {
  const [name, setName] = useState("");

```

```

return (
  <form>

    <input

      type="text"

      value={name}

      onChange={(e) => setName(e.target.value)}

    />

    <p>You typed: {name}</p>

  </form>

);
}

```

React Router Basics

React Router enables navigation between pages without reloading the browser.

Example Router Usage

```

import { BrowserRouter, Routes, Route } from "react-router-dom";

<BrowserRouter>

  <Routes>

    <Route path="/" element={<Home />} />

    <Route path="/about" element={<About />} />

  </Routes></BrowserRouter>

```

Introduction to Node.js

Today's session focused on **Node.js**, an essential technology in the MERN stack for building backend applications. While JavaScript was originally designed to run in the browser, Node.js extended its capabilities, enabling JavaScript to run on servers as well. This shift marked a major evolution in web development and laid the foundation for modern full-stack JavaScript.

The instructor highlighted that Node.js allows developers to handle server-side programming, APIs, authentication, data processing, and file management, all using a single language — **JavaScript**. This makes MERN extremely efficient for development teams.

What is Node.js?

Node.js is an **open-source, cross-platform, JavaScript runtime environment** built on Google Chrome's V8 engine. Unlike traditional backends that rely heavily on multithreading, Node.js uses an event-driven, non-blocking I/O model, making it highly scalable for real-time applications.

Common use cases of Node.js:

- Backend APIs
- Real-time chat applications
- Streaming platforms
- File system utilities
- Microservices

Node.js Architecture & Event Loop

Node.js uses a **single-threaded event-driven architecture**. Instead of creating multiple threads, Node.js manages tasks using the **Event Loop**, which handles asynchronous operations efficiently.

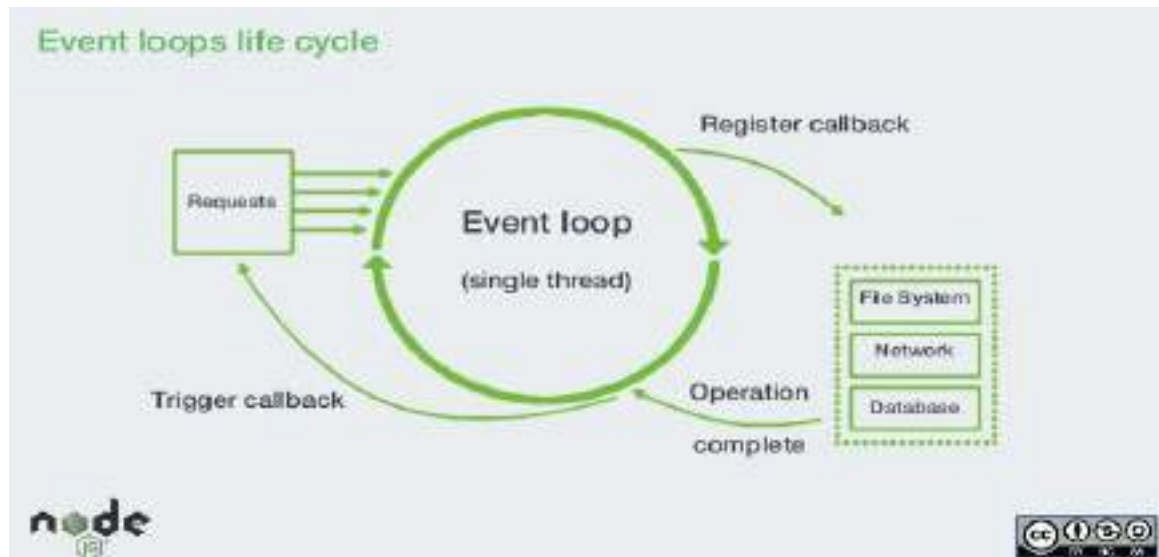


Figure 06: Node.js Event Loop Working

This structure helps Node.js handle thousands of simultaneous requests without performance issues.

Node Package Manager (NPM)

NPM is the default package manager for Node.js and one of the largest package ecosystems in the world.

I learned how to:

- Initialize a project using `npm init`
- Install packages like Express, Mongoose, Nodemon
- Manage dependencies via `package.json`

Example commands:

```
npm init -y
```

```
npm install express
```

```
npm install mongoose
```

Creating a Simple Node.js Server

The instructor demonstrated how to set up a basic HTTP server using Node's built-in http module. This helped me understand how backend communication works at a low level.

Code Example: Basic Node Server

```
const http = require('http');

const server = http.createServer((req, res) => {

  res.writeHead(200, { 'Content-Type': 'text/plain' });

  res.end('Hello from Node.js Server');

});

server.listen(5000, () => {

  console.log('Server is running on port 5000');

});
```

This simple server responds with a message when accessed through the browser.

Working with File System (fs Module)

Node.js provides built-in modules like **fs** to interact with files. I practiced reading and writing files:

Example Code: Writing to a File

```
const fs = require('fs');

fs.writeFileSync('message.txt', 'This is my first Node program!');
```

Example Code: Reading a File

```
const data = fs.readFileSync('message.txt', 'utf8');  
  
console.log(data);
```

These operations helped me understand how backend systems manage logs, configuration files, and user data.

package.json Configuration

The package.json file stores metadata about the project such as:

- Project name and version

- Dependencies and dev-dependencies

- Scripts (e.g., "start": "node index.js")

Example:

```
{  
  
  "name": "my-node-app",  
  
  "version": "1.0.0",  
  
  "scripts": {  
  
    "start": "node index.js"  
  
  }  
}
```

This file is essential for deploying MERN applications on cloud platforms.

Chapter 5

Express.js Framework and REST API Development

Today, I was introduced to **Express.js**, the lightweight and flexible Node.js framework widely used for backend development in MERN applications. While Node.js provides the base environment, Express simplifies routing, middleware usage, API creation, and handling server logic, making backend development efficient and organized.

Express.js is the backbone of most modern JavaScript backends, offering a clean structure for developing scalable REST APIs that communicate with frontend applications like React.

Introduction to Express.js

Express.js is a **minimal and unopinionated web framework** built on top of Node.js. It provides an easier way to create servers, manage routes, process requests, and send responses.

Key features of Express:

- Simple and powerful routing
- Built-in and custom middleware
- JSON request handling
- Ability to integrate with MongoDB through Mongoose
- Highly scalable and flexible

This framework is essential for building the API layer in MERN projects.

Routing in Express.js

Express routing allows the backend to respond to specific URL paths with defined methods (GET, POST, PUT, DELETE). Routing determines how the server handles different client requests.

Example of basic routing:

```
const express = require('express');const app = express();

app.get('/', (req, res) => {

  res.send('Welcome to Express Server');

});

app.listen(5000, () => {

  console.log('Server running on port 5000');

});
```

Concept of Middleware

Middleware functions in Express process requests **before** they reach the endpoint. They are used for Logging requests, Parsing JSON data, Authentication and Error handling.

Example of a simple middleware:

```
app.use((req, res, next) => {

  console.log('Request received:', req.method, req.url);

  next();

});
```

Express also provides built-in middleware like:

`express.json()` for parsing JSON

`express.urlencoded()` for form data

Creating REST API with Express.js

A REST API (Representational State Transfer) allows communication between the frontend and backend using HTTP methods.

Example: Basic CRUD API

```
app.get('/users', (req, res) => {  
  res.json({ message: "Get all users" });  
});  
  
app.post('/users', (req, res) => {  
  res.json({ message: "Create a new user" });  
});  
  
app.put('/users/:id', (req, res) => {  
  res.json({ message: `Update user ${req.params.id}` });  
});  
  
app.delete('/users/:id', (req, res) => {  
  res.json({ message: `Delete user ${req.params.id}` });  
});
```

This structure forms the backbone of all backend operations in MERN apps.

Error Handling in Express

Express provides structured error handling to manage unexpected issues.

Example:

```
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).send("Something went wrong!");  
});
```

Good error handling ensures application stability and better debugging.

Express Folder Structure

A clean folder structure keeps routes, controllers, models, middleware organized.

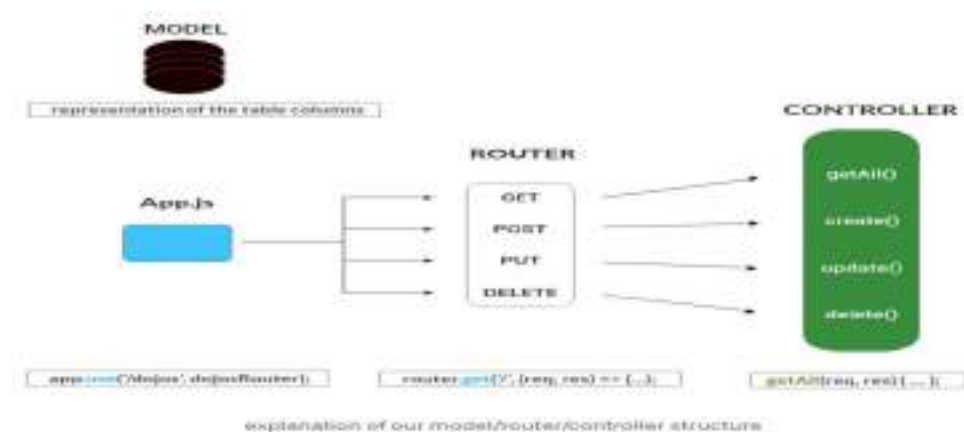


Figure 07: Express.js Basic Folder Structure

A typical Express project structure:

/project /route /controllers /models /middleware /config server.js

API Example Code Snippet

Later in the session, I created a small example API:

```
const express = require('express');const app = express();

app.use(express.json());

app.post('/register', (req, res) => {

  const { name, email } = req.body;

  res.json({ message: "User registered", data: { name, email } });

});

app.listen(4000, () => {

  console.log('API running on port 4000');

});
```

This helped me understand how data flows from the client-side form to the server.

MongoDB & Mongoose

Today my industrial training focused on the database layer of the MERN stack—**MongoDB and Mongoose**. Databases are an essential part of any full-stack application, and MongoDB is widely used due to its flexibility, scalability, and JSON-like document structure. Mongoose acts as an elegant Object Data Modeling (ODM) library that simplifies interactions between Node.js and MongoDB.

Understanding how data is stored, retrieved, and managed is crucial for building dynamic applications such as e-commerce, social networks, dashboards, and more.

Introduction to NoSQL Databases

The instructor explained the difference between SQL and NoSQL databases.

Unlike SQL tables with rows and columns, NoSQL databases store data in a flexible, schema-less document-based structure.

Benefits of NoSQL (MongoDB):

- Schema flexibility
- Faster read/write operations
- Easily scalable
- JSON-like documents make it ideal for JavaScript applications

MongoDB stores data in **documents**, which are grouped into **collections**.

Document Structure in MongoDB

A MongoDB document is similar to a JSON object but stored in **BSON** (Binary JSON) format.

Typical structure:

```
{
```

```
"name": "Rahul Sharma",  
"email": "rahul@example.com",  
"age": 22,  
"skills": ["HTML", "React", "Node"]}
```

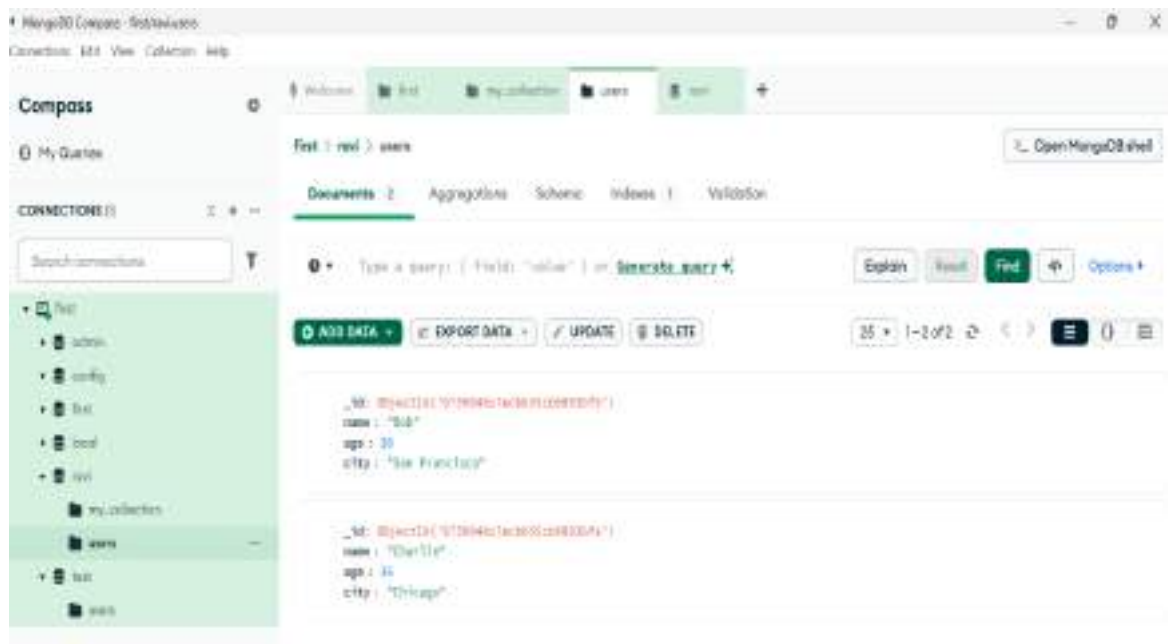


Figure 08: MongoDB Document Structure Overview

This visual representation helped me understand how MongoDB organizes and stores data.

CRUD Operations in MongoDB

MongoDB supports the four primary operations:

Create (insert)

Read (find)

Update (modify)

Delete (remove)

Basic examples:

```
db.users.insertOne({ name: "Raj", age: 21 });

db.users.find();

db.users.updateOne({ name: "Raj" }, { $set: { age: 22 } });

db.users.deleteOne({ name: "Raj" });
```

These commands form the core of data manipulation in MongoDB.

Introduction to Mongoose

Mongoose is an ODM library that helps structure MongoDB data using **schemas** and **models**.

Instead of handling raw MongoDB queries, Mongoose provides a clean, object-oriented approach.

Example: Creating a Mongoose Schema

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({

  name: String,

  email: String,

  age: Number

});

module.exports = mongoose.model('User', userSchema);
```

Mongoose Model & Operations (CRUD Example)

After defining a schema, we can perform CRUD operations using the model:

```
const User = require('./models/user');

// CREATEawait User.create({ name: "Kanchan", email: "K@gmail.com" });

// READconst users = await User.find();
```

```
// UPDATEawait User.updateOne({ name: "Kanchan" }, { age: 20 });

// DELETEawait User.deleteOne({ name: "Kanchan" });
```

This demonstrates how Mongoose simplifies database operations.

SQL vs NoSQL (MongoDB) Comparison

Feature	SQL Database	NoSQL (MongoDB)
Structure	Tables with rows/columns	Collections with flexible document
Schema	Fixed	Dynamic / Schema-less
Data Format	Structured	JSON/BSON
Scalability	Vertical scaling	Horizontal scaling
Ideal Use Case	Finance, ERP	MERN apps, real-time apps

Table 03: SQL vs NoSQL (MongoDB) Differences

This table helped me understand why MongoDB fits MERN better.

Relationships in MongoDB (Embedding vs Referencing)

MongoDB supports two types of relationships:

1. Embedding

Storing related documents inside one main document.

Example:

```
{ "name": "Aman",
  "address": { "city": "Jaipur", "pincode": 302022 } }
```

2. Referencing

Storing document IDs to link separate collections.

Example:

```
{  "user": "65ae99f23bcd1",  
  "product": "9348abc924567" }
```

Both approaches are used depending on application needs.

Database Connectivity Using Mongoose

Connecting Node.js to MongoDB Atlas / local database:

```
mongoose.connect('mongodb://localhost:27017/mern', {  
  useNewUrlParser: true,  
  useUnifiedTopology: true  
})  
  
.then(() => console.log("Database Connected"))  
  
.catch(err => console.log(err));
```

This connection enables Express.js to perform database operations.

MERN Integration and Authentication, Security & Deployment

In last few days of my industrial training focused on integrating the entire MERN stack and implementing authentication, security, and deployment processes. These concepts helped me understand how full-stack applications function from user interaction to database storage, and how to secure and deploy them for real-world usage. In the last few days of my internship, due to limited time, I was able to gain only an overview of MERN Integration, Authentication, Security, and Deployment, but it provided me a strong foundation for continuing full-stack development.

MERN Integration Overview

MERN integration connects:

React.js (Frontend)

Express.js + Node.js (Backend/API Layer)

MongoDB (Database Layer)

The instructor explained how a full MERN application enables users to submit data, fetch data, authenticate, route between pages, and store information in a secure and scalable database.

React to Express Communication

React communicates with the backend through API calls using Axios or Fetch.

Example (GET request in React):

```
axios.get("http://localhost:5000/api/users")  
  
  .then(res => console.log(res.data));
```

Express receives the request, processes it, and interacts with MongoDB.

Express API Endpoint (Backend)

A simple POST API endpoint to create a new user:

```
app.post("/api/create-user", async (req, res) => {  
  
  try {  
  
    const user = await User.create(req.body);  
  
    res.json({ message: "User created", user });  
  
  } catch (err) {  
  
    res.status(500).json({ error: "Failed" });  
  
  }  
  
});
```

This endpoint stores data using Mongoose models.

Sending Data from React to Backend

```
const [name, setName] = useState("");  
  
const handleSubmit = () => {  
  
  axios.post("http://localhost:5000/api/create-user", { name });  
  
};
```

This taught me how React forms communicate with backend routes.

Fetching MongoDB Data in React

```
useEffect(() => {  
  
  axios.get("http://localhost:5000/api/users")  
  
    .then(res => setUsers(res.data));  
  
}, []);
```

React automatically updates the UI when state changes.

Full MERN Data Flow

React collects user input

React → Express API call

Express validates request

Express → Mongoose → MongoDB

MongoDB sends response

Express sends JSON to React

React UI updates dynamically

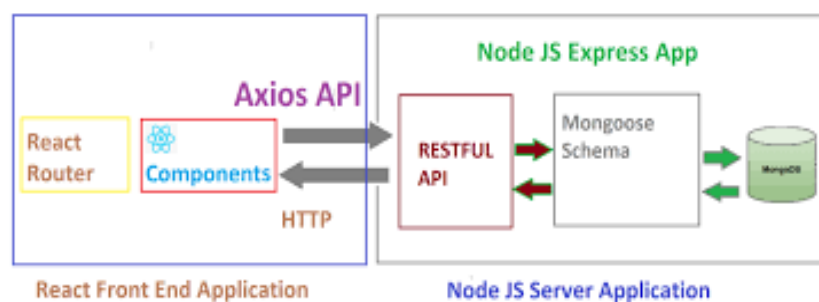


Figure 09: MERN CRUD Data Flow

Folder Structure for MERN Projects

/client → React frontend

/server → Express backend

/models → Mongoose schemas

/routes → API routes

/config → Database configuration

This organization keeps the project maintainable.

Authentication, Security & Deployment

I learned how authentication, security, and deployment convert a basic MERN project into a production-ready web application.

JWT Authentication

JWT (JSON Web Token) is used to securely verify user identity. The backend generates a token after login, and the frontend stores it (usually in localStorage).

Example: Generating JWT Token

```
const token = jwt.sign({ id: user._id }, "secretkey", { expiresIn: "1d" });
```

Password Hashing with bcrypt

To secure user credentials, passwords are hashed before storing them.

```
const bcrypt = require('bcrypt');  
  
const hash = await bcrypt.hash(password, 10);
```

This ensures raw passwords are never stored in the database.

Protected Routes (Backend Security)

Protected routes ensure only logged-in users can access specific APIs.

```
function authMiddleware(req, res, next) {  
  const token = req.headers.authorization;  
  
  if (!token) return res.status(401).send("Unauthorized");  
  
  jwt.verify(token, "secretkey", (err, user) => {  
    if (err) return res.status(403).send("Invalid Token");  
  
    req.user = user;  
  
    next();  
  });  
}
```

```
});  
  
}
```

Common Security Practices

The trainer introduced important backend security tools:

CORS → Controls which frontend can access API

Helmet → Secures HTTP headers

Rate Limiting → Prevents abuse and excessive requests

Environment Variables (.env) → Hide sensitive data like DB URLs & keys

These practices prevent common attacks like CSRF, XSS, and brute force attempts.

Deployment Overview

Deployment prepares the application for public use. We explored:

- **Frontend Deployment Options**

Netlify

Vercel

- **Backend Deployment Options**

Render

Railway

DigitalOcean

- **Database Deployment**

MongoDB Atlas (Cloud-based MongoDB service)

Deployment Process (Simplified):

Upload React build folder to Vercel/Netlify

Upload Express backend to Render

Connect MongoDB Atlas URL in backend .env

Set environment variables

Test final deployment in browser

Chapter 8

Objectives

- Provide a single, searchable source of RTU notes, PYQs and syllabus.
- Make materials downloadable as PDFs and link to lecture videos for richer study.
- Maintain a consistent information hierarchy: Year → Branch → Subject → Unit.
- Implement a clean, accessible UI with light/dark themes and mobile responsiveness.
- Keep the architecture simple (React + JSON) so non-developers can update content.

Key Features

- **Hierarchical Navigation:** Homepage presents Year cards; subsequent screens let users choose Branch → Subject → Notes/PYQ/Syllabus.
- **Content Types:** Unit PDFs, syllabus downloads, midterm PYQs, main exam PYQs, and YouTube lecture links.
- **Theme Toggle:** Light mode (Blue, White, Yellow) and Dark mode (Navy, Charcoal, Cyan).
- **Mobile-first, Responsive Design:** Works on phones and desktops with fluid layouts and accessible controls.
- **Lightweight Data Store:** JSON files (per-year / per-branch) store metadata and file URLs.
- **About & Contact Pages:** Project information and a simple contact form or mailto link.

System Architecture & File Structure

High-level: React front-end (Create React App / Vite) → Static JSON files → Public pdfs/ folder for PDFs and assets/ for images.

Suggested file structure:

```
rtupedia/  
  
├─ public/  
  
|   ├─ pdfs/  
  
|   |   ├─ 1st_year/  
  
|   |   └─ syllabus/  
  
|   └─ index.html  
  
├─ src/  
  
|   ├─ components/  
  
|   |   ├─ YearCard.jsx  
  
|   |   ├─ BranchList.jsx  
  
|   |   ├─ SubjectView.jsx  
  
|   |   └─ ThemeToggle.jsx  
  
|   ├─ pages/  
  
|   |   ├─ Home.jsx  
  
|   |   ├─ NotesPage.jsx  
  
|   |   └─ About.jsx  
  
|   └─ data/  
  
|       ├─ rtu-index.json  
  
|       └─ year1-cse.json  
  
└─ App.jsx  
  
└─ index.jsx  
  
└─ package.json  
  
└─ README.md
```

Data Model (JSON)

Each JSON file stores an array of branches, subjects and units. Example entry:

```
{
  "year": "1st",
  "branch": "CSE",
  "subjects": [
    {
      "code": "1FY2-01",
      "name": "Engineering Mathematics-I",
      "units": [
        {
          "unit": "Calculus", "pdf": "/pdfs/1st/cse/math_unit1.pdf", "video":
          "https://youtube.com/..."
        }
      ],
      "pyq": {
        "mid": "/pdfs/pyq/1st/cse/math_mid.pdf",
        "main": "/pdfs/pyq/1st/cse/math_main.pdf"
      }
    }
  ]
}
```

This approach keeps front-end logic simple: fetch JSON, render lists, and link PDFs.

Implementation Notes

Framework: React (functional components + hooks).

Routing: React Router for page navigation (Home → Branch → Subject → Unit).

State management: Local component state and context for theme/settings; JSON-driven content removal of complex state libraries.

PDF Handling: PDFs are stored in public/pdfs and referenced by relative URLs so they are directly downloadable.

Media Links: Lecture references are external YouTube URLs opened in new tabs.

UI / UX

- Homepage uses Year cards (visual tiles). Selecting a year reveals branches as smaller cards or a responsive grid.
- Subject pages display Notes, PYQs, and Syllabus tabs/buttons. Each unit appears as a card with a short description, Download PDF button, and Lecture link.
- Theme toggle sits in the header; theme preference is saved to localStorage to persist between sessions.

Responsiveness & Accessibility

- Layout is mobile-first; CSS Grid and Flexbox handle reflow across breakpoints.
- Buttons and links are large enough for touch targets; semantic HTML and aria-* attributes provide better screen reader support.

Testing & Validation

- Manual cross-device testing (Chrome desktop, mobile viewport, and a mid-range Android phone).
- Validate JSON schema for required fields (year, branch, subject, unit, pdf) before rendering to avoid broken links.

Deployment

- Since the app is static (React build + static PDFs), it can be deployed to GitHub Pages, Netlify, or Vercel.
- Build step: npm run build → upload build/ directory along with public/pdfs to the hosting provider.

Security & Copyright

- Ensure PDFs uploaded are owned or properly permitted to be distributed.
- Avoid storing sensitive information in JSON files in the public folder; these are static, public assets.

Future Enhancements

- Add search across subjects/units and keyword-based filtering.
- Implement user accounts to save favourites and notes.
- Convert JSON to a small CMS (e.g., Firebase or headless CMS) for easier remote updates.
- Add analytics to track popular subjects and frequently downloaded files.

Executive Summary

RTUpedia is a responsive React-based web application designed to centralize and serve Rajasthan Technical University (RTU) study materials for B.Tech students across all branches and semesters. The site organizes content by Year → Branch → Subject → Units and exposes downloadable PDFs, syllabus files, past year question (PYQ) papers (mid and main), and reference lecture video links. The system uses JSON files as a lightweight content store for metadata and file references, enabling easy content addition without a heavy backend.

Conclusion

RTUpedia provides a focused, lightweight solution to centralize RTU academic resources. Its JSON-driven architecture keeps content maintenance simple while React ensures a fast, interactive UI across devices. With modest enhancements (search, CMS), RTUpedia can grow into a full-featured study portal for RTU students.

Snapshots:



Snapshot 1



Snapshot 2



Snapshot 3

Training Summary

My industrial training in Web Development using the MERN Stack provided a strong foundation in both frontend and backend development. I started with the basics of HTML, CSS, and JavaScript, which helped me understand how web pages are structured, styled, and made interactive. I then progressed to React.js, where I learned component-based development, hooks, routing, form handling, and API integration.

On the backend side, I worked with Node.js and Express.js to build RESTful APIs, manage routes, handle middleware, and process client requests. I also gained hands-on experience with MongoDB and Mongoose for designing schemas, performing CRUD operations, and managing database connectivity.

Later, I explored full MERN integration, where the frontend, backend, and database were connected into a complete workflow. I also studied essential concepts like JWT authentication, password hashing, protected routes, environment variables, and basic security measures. Additionally, I learned the deployment process using platforms such as Render, Vercel, and MongoDB Atlas.

As part of the training, I developed a small **MERN Task Manager project**, which helped me practically apply everything I learned—from UI design and API communication to database operations.

Overall, this training strengthened my technical skills, improved my understanding of real-world development practices, and boosted my confidence in building and managing full-stack applications. It gave me the necessary exposure to continue progressing as a full-stack developer.

Key Outcomes

During this training, I achieved the following outcomes:

- Understood the fundamentals of HTML, CSS, and JavaScript.
- Learned React components, hooks, routing, and API integration.
- Built backend APIs using Node.js and Express.js.
- Performed CRUD operations using MongoDB and Mongoose.
- Understood full MERN integration and data flow across layers.
- Gained basic knowledge of authentication (JWT) and security practices.
- Developed a small MERN Task Manager project applying all core concepts.

Next Steps

To continue improving, I plan to:

1. Learn advanced React concepts and state management tools.
2. Build more MERN projects for practical experience.
3. Explore advanced MongoDB, authentication, and deployment techniques.
4. Learn TypeScript and modern UI frameworks to improve code quality.

References

- **MongoDB Documentation** – Official Guide
<https://www.mongodb.com/docs/>
- **Mongoose Documentation** – ODM for Node.js
<https://mongoosejs.com/docs/>
- **Express.js Documentation** – Web Framework for Node.js
<https://expressjs.com/>
- **Node.js Documentation** – JavaScript Runtime Environment
<https://nodejs.org/en/docs/>
- **React.js Official Documentation** – Frontend Library
<https://react.dev/>
(Formerly <https://reactjs.org/>)
- **HTML, CSS & JavaScript Guides** - Mozilla Developer Network (MDN)
<https://developer.mozilla.org/>
- **Web Development Tutorials** - W3Schools
<https://www.w3schools.com/>
- **Full-Stack Web Development Resources** - FreeCodeCamp
<https://www.freecodecamp.org/>
- **MERN Stack Articles** - GeeksforGeeks
<https://www.geeksforgeeks.org/>
- **Community Discussions and Error Solutions** - Stack Overflow
<https://stackoverflow.com/>